BioSpecInfo

Riferimento delle interfacce — BioSpecInfo

Samuele Pio Provenzano

Versione **bsi-v200** · commit `3bcf3b9`

Generato il 2026-10-14

Documento generato automaticamente dal sorgente Markdown versionato nel repository. La versione e il commit stampati in ogni piè di pagina identificano esattamente lo stato del software descritto.

Riferimento delle interfacce — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Versione descritta	`bsi-v200`
Scopo	Documentare le interfacce HTTP del progetto e le API esterne consumate dall'applicazione.

0. Premessa necessaria

BioSpecInfo non espone API. È una Progressive Web App interamente client-side, servita come file statici: non esiste un backend, non esiste un server applicativo, non esistono endpoint REST del prodotto.

Questo documento sarebbe quindi vuoto, se non fosse per due interfacce che esistono davvero e che un valutatore ha diritto di vedere documentate:

	Cosa è	Documentato in
Proxy Spectra	Un Cloudflare Worker **opzionale**, di proprietà dell'utente, che inoltra le chiamate ai fornitori AI. È l'unica interfaccia HTTP scritta in questo progetto.	§1, §2
API esterne consumate	Servizi pubblici di terzi che l'applicazione interroga, sempre in modo opzionale.	§3
API interne JavaScript	I punti di estensione del codice, per chi deve intervenirci.	§4

> Una specifica OpenAPI di endpoint inesistenti sarebbe documentazione plausibile e falsa. Qui è descritto soltanto ciò che esiste.

1. Proxy Spectra — panoramica

Il proxy risolve due problemi che dal browser non hanno altra soluzione:

1. **CORS.** Diversi fornitori AI non inviano le intestazioni che permettono a

una pagina web di chiamarli direttamente. Dal Worker la chiamata parte da un server e il vincolo non si applica.

1. **Custodia delle chiavi.** Con il proxy la chiave API risiede fra i segreti

del Worker e non sul dispositivo dell'utente.

È **opzionale**: senza proxy l'applicazione funziona usando le chiavi inserite localmente dall'utente.

Deploy

```
cd proxy
npx wrangler login
npx wrangler deploy
npx wrangler secret put GROQ_KEYS     # una o più chiavi, separate da virgola
```

Configurazione

Variabile	Tipo	Default	Descrizione
`ORIGINI`	segreto/variabile	*(vuoto = tutte)*	Elenco di origini ammesse, separate da virgola. **Lasciarlo vuoto rende il proxy aperto a chiunque:** sconsigliato.
`LIMITE_IP`	variabile	`20`	Richieste per singolo IP nella finestra temporale
`TETTO_GIORNO`	variabile	`2000`	Richieste totali giornaliere
`GROQ_KEYS`, `GEMINI_KEYS`, `ANTHROPIC_KEYS`, `OPENAI_KEYS`, `XAI_KEYS`, `NVIDIA_KEYS`, `ZAI_KEYS`, `DEEPSEEK_KEYS`	segreti	—	Chiavi API. Più chiavi separate da virgola: esaurita la quota della prima (429), il proxy passa alla successiva **nella stessa richiesta**.

Endpoint

Metodo	Percorso	Descrizione
`OPTIONS`	`/*`	Preflight CORS
`GET`	`/stato`	Quali fornitori hanno una chiave configurata
`GET` `POST`	`/{fornitore}/{percorso...}`	Inoltro al fornitore

Fornitori riconosciuti: `anthropic` · `gemini` · `groq` · `xai` · `nvidia` · `zai` · `openai` · `deepseek`

Codici di stato

Codice	Significato
`200`	Risposta del fornitore, inoltrata così com'è
`204`	Preflight CORS accettato
`400`	Corpo della richiesta illeggibile
`403`	Origine non ammessa dall'elenco `ORIGINI`
`404`	Fornitore sconosciuto
`405`	Metodo diverso da GET, POST o OPTIONS
`429`	Limite per IP o tetto giornaliero superato

Codice	Significato
`503`	Nessuna chiave configurata per quel fornitore

Scelte di sicurezza

- **Nessuna intestazione inoltrata ciecamente.** Passano solo

`anthropic-version` e `anthropic-beta`; qualunque credenziale inviata dal client resta fuori. Un proxy che inoltra tutto è un relay aperto.

- **Il parametro `key` non viene propagato.** Se un client lo mette in query

string, il proxy lo scarta: la chiave è quella del server.

- **Il corpo viene letto una volta sola e conservato**, perché per ritentare con

una chiave diversa occorre poterlo rimandare — e uno stream si consuma.

---

2. Specifica OpenAPI 3.0

```
openapi: 3.0.3
info:
  title: Proxy Spectra
  version: "1.0.0"
  description: |
    Proxy opzionale (Cloudflare Worker) per le chiamate ai fornitori di modelli
    linguistici. Elimina i blocchi CORS e mantiene le chiavi API sul server
    anziché sul dispositivo dell'utente.

    Non fa parte dell'applicazione BioSpecInfo: viene pubblicato dall'utente sul
    proprio account Cloudflare, e l'applicazione lo usa solo se ne conosce
    l'indirizzo.
  license:
    name: Proprietary — All rights reserved (Samuele Pio Provenzano)
servers:
  - url: https://{nomeworker}.{account}.workers.dev
    description: Istanza dell'utente
    variables:
      nomeworker: { default: spectra-proxy }
      account:    { default: esempio }

tags:
  - name: Stato
    description: Interrogazione della configurazione, senza rivelare segreti
  - name: Inoltro
    description: Inoltro delle richieste ai fornitori di modelli

paths:
  /stato:
    get:
      tags: [Stato]
      summary: Fornitori configurati
      description: |
        Elenca i fornitori per i quali il proxy possiede almeno una chiave.
        Non rivela alcuna chiave. L'applicazione lo usa per mostrare soltanto i
        modelli che funzionano davvero, invece di farli fallire uno per uno.
      responses:
        "200":
          description: Elenco dei fornitori attivi
          content:
            application/json:
              schema:
                type: object
                required: [fornitori]
                properties:
                  fornitori:
                    type: array
                    items:
                      type: string
                      enum: [anthropic, gemini, groq, xai, nvidia, zai, openai, deepseek]
              examples:
                dueChiavi:
                  summary: Due fornitori configurati
                  value: { fornitori: [groq, gemini] }
                nessuna:
                  summary: Worker pubblicato ma senza segreti
                  value: { fornitori: [] }
```

```
      "403":
        $ref: "#/components/responses/OrigineNonAmmessa"

  /{fornitore}/{percorso}:
    parameters:
      - name: fornitore
        in: path
        required: true
        description: Fornitore di destinazione
        schema:
          type: string
          enum: [anthropic, gemini, groq, xai, nvidia, zai, openai, deepseek]
      - name: percorso
        in: path
        required: true
        description: |
          Percorso dell'endpoint presso il fornitore, ricomposto così com'è.
          Esempi: `openai/v1/chat/completions` per Groq,
          `v1beta/models/gemini-3.6-flash:streamGenerateContent` per Gemini.
        schema: { type: string }
        example: openai/v1/chat/completions
    post:
      tags: [Inoltro]
      summary: Inoltra una richiesta al fornitore
      description: |
        Il corpo viene trasmesso invariato. Il proxy aggiunge la chiave nel modo
        richiesto da quel fornitore (intestazione `Authorization: Bearer`,
        `x-api-key`, oppure parametro in query) e, se la chiamata fallisce per
        quota esaurita, ritenta con la chiave successiva nella stessa richiesta.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              description: Corpo previsto dall'API del fornitore, trasmesso invariato
            examples:
              groq:
                summary: Completamento in streaming su Groq
                value:
                  model: llama-3.3-70b-versatile
                  stream: true
                  messages:
                    - role: user
                      content: Quali bande IR identificano un acido carbossilico?
      responses:
        "200":
          description: |
            Risposta del fornitore, inoltrata senza modifiche (tipicamente
            `text/event-stream` quando `stream: true`).
          content:
            application/json:
              schema: { type: object }
            text/event-stream:
              schema: { type: string }
        "400": { $ref: "#/components/responses/CorpoIlleggibile" }
        "403": { $ref: "#/components/responses/OrigineNonAmmessa" }
        "404": { $ref: "#/components/responses/FornitoreSconosciuto" }
        "429": { $ref: "#/components/responses/TroppeRichieste" }
        "503": { $ref: "#/components/responses/ChiaveAssente" }
```

```
    get:
      tags: [Inoltro]
      summary: Inoltra una richiesta in lettura al fornitore
      description: |
        Serve, fra l'altro, alla risoluzione del modello Gemini, che interroga
        `ListModels` in GET e senza chiave nel browser non potrebbe passare.
      responses:
        "200":
          description: Risposta del fornitore
          content:
            application/json:
              schema: { type: object }
        "403": { $ref: "#/components/responses/OrigineNonAmmessa" }
        "404": { $ref: "#/components/responses/FornitoreSconosciuto" }
        "429": { $ref: "#/components/responses/TroppeRichieste" }
        "503": { $ref: "#/components/responses/ChiaveAssente" }

components:
  schemas:
    Errore:
      type: object
      required: [errore]
      properties:
        errore:
          type: string
          description: Messaggio in italiano, pensato per essere mostrato all'utente
  responses:
    OrigineNonAmmessa:
      description: L'origine della richiesta non è nell'elenco `ORIGINI`
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Origine non ammessa da questo proxy." }
    FornitoreSconosciuto:
      description: Il primo segmento del percorso non corrisponde a un fornitore noto
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Fornitore sconosciuto: pippo" }
    CorpoIlleggibile:
      description: Il corpo della richiesta non è stato leggibile
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Corpo della richiesta illeggibile." }
    TroppeRichieste:
      description: Superato il limite per IP o il tetto giornaliero
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Troppe richieste da questo indirizzo: riprova fra un minuto." }
    ChiaveAssente:
      description: Il proxy non ha una chiave configurata per quel fornitore
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example:
            errore: "Il proxy non ha una chiave per groq. Impostala con: wrangler secret put
GROQ_KEYS"
```

```
security: []
# L'autenticazione del CLIENT è volutamente assente: il controllo di accesso è
# l'elenco ORIGINI più i limiti per IP. Aggiungere un token lato client
# significherebbe scriverlo in una pagina statica, cioè renderlo pubblico.
```

3. API esterne consumate dall'applicazione

Tutte **opzionali**: senza rete l'applicazione resta pienamente funzionante sui dati locali, e lo dichiara all'utente.

Servizio	Endpoint usati	A cosa serve	Se non risponde

PubChem PUG-REST	`/compound/name/{nome}/property/...`
	 `/compound/smiles/{smiles}/SDF?record_type=3d`
	 `/compound/cid/{cid}/PNG`
		Nome IUPAC, formula, CAS, pittogrammi GHS, conformeri 3D, immagini 2D	L'app usa i dati tabulati locali e lo segnala

RCSB PDB	`https://files.rcsb.org/download/{pdb}.pdb`	Strutture proteiche per il visore 3D	Messaggio esplicito nel visore
NASA / ESA	Immagini e dati pubblici	Modulo di astrochimica	Il modulo mostra i dati tabulati
Fornitori AI	A scelta dell'utente, o via proxy	Risposte dell'assistente	L'assistente lo dichiara e propone un'alternativa raggiungibile

Nessuna di queste chiamate trasmette dati personali. Non sono presenti identificativi persistenti né telemetria attiva (`BSI_TELEMETRY_URL` è vuoto).

4. API interne JavaScript

Punti di estensione per chi deve intervenire sul codice. Sono globali sul `window` perché l'applicazione è un insieme di script classici, non moduli ES.

`window.BSISpettri` — motore spettrale

Funzione	Firma	Descrizione
`gruppi`	`(smiles, callback)`	Riconosce i gruppi funzionali via SMARTS/RDKit. Richiama con `null` se RDKit non è disponibile: il chiamante decide se ripiegare o dichiarare che non può calcolare.
`gruppiSync`	`(smiles) → oggetto | null`	Versione sincrona: risponde solo se la molecola è già in cache
`bandeDaGruppi`	`(gruppi) → banda[]`	Tabella delle bande IR `{n, pk, w, i, f}`

Funzione	Firma	Descrizione
`svgIR`	`(bande, opzioni) → string`	Spettro IR in trasmittanza, SVG, deterministico
`legendaIR`	`(bande) → string`	Assegnazioni in HTML
`quandoPronto`	`(callback)`	Notifica quando l'analisi di una molecola è disponibile, per ridisegnare
`precarica`	`()`	Scalda RDKit senza che nessuno stia aspettando

`window.bsiProxy` — stato del proxy

Funzione	Descrizione
`url()`	Indirizzo configurato, stringa vuota se assente
`stato()`	Promessa: interroga `/stato` e memorizza i fornitori coperti
`copre(idFornitore)`	Se quel fornitore è coperto dal proxy
`ricarica()`	Rilegge la configurazione

Altri punti notevoli

Simbolo	Descrizione
`window.bsiLoadRDKit(cb)`	Carica RDKit una volta sola e richiama con l'istanza
`window.irBandList(smiles)`	Bande IR — sostituita dal motore nuovo, mantiene la firma originale
`window.makeIRsvg(bande, colore)`	Disegno IR — idem
`window.nmrPeakList(smiles)`	Picchi ^{1}H con integrazione
`window.bsiCancellaDati(gruppi)`	Cancellazione selettiva dei dati salvati
`window.bsiOpenUpdate()`	Apre la finestra degli aggiornamenti

Documento aggiornato alla versione `bsi-v200`.